

Dokumentasi Perhitungan Fuel Ratio

Source of truth formula berada pada `backend/app/services/` dan `backend/app/services/monitoring_service.py`.

Konvensi dan target SPO

Simbol	Arti	Satuan
qty	Jumlah unit	unit
fuel_cons	Konsumsi fuel	L/jam atau L sesuai konteks
productivity	Produktivitas	BCM/jam
PA	Physical Availability	0–1
UA	Use of Availability	0–1
EWB	Effective Working Hours	jam
FR	Fuel Ratio	L/BCM

Aktivitas	Target SPO FR
Loading	0.1325
Hauling	0.6007
Supporting	0.1879
Dewatering	0.3233

Loading

Reference

```
fuel_cons_reference = equipment.qty × fuel_reference.average
productivity = equipment.qty × equipment.productivity
fuel_ratio_reference = round(fuel_cons_reference / productivity, 2)
```

Equipment dan fuel reference harus memiliki activity `Loading`. Quantity dan productivity harus lebih besar dari 0.

Actual operational

Jika `fuel_consumed_liters` dan `operating_hours` keduanya tersedia dan lebih besar dari 0:

```
fuel_cons_actual = fuel_consumed_liters / operating_hours
fuel_ratio_actual = round(fuel_cons_actual / productivity, 2)
```

```
fuel_cons = fuel_cons_actual
fuel_ratio = fuel_ratio_actual
data_source = OPERATIONAL_ACTUAL
```

Jika actual tidak lengkap atau tidak valid, sistem memakai reference:

```
fuel_cons = fuel_cons_reference
fuel_ratio = fuel_ratio_reference
data_source = OEM_REFERENCE
```

Batch

```
total_fuel =  $\Sigma(\text{qty} \times \text{fuel\_cons})$ 
total_productivity =  $\Sigma(\text{qty} \times \text{productivity})$ 
fuel_ratio = round(total_fuel / total_productivity, 2)
```

rows harus berisi minimal satu item. Quantity, fuel consumption, dan productivity harus lebih besar dari 0.

Hauling

Engine mencari **bcm_per_hr** dari **hauling_distance_ref** berdasarkan **distance_km**.

Jika tidak ditemukan:

1. gunakan **equipment.productivity** jika nilainya lebih besar dari 0;
2. jika tidak, gunakan fallback **110.0 BCM/hr**.

```
fuel_cons = equipment.qty × fuel_reference.average
productivity = equipment.qty × bcm_per_hr
fuel_ratio = round(fuel_cons / productivity, 2)
```

distance_km harus lebih besar dari 0 dan productivity akhir tidak boleh 0.

Supporting

Reference

```
fuel_cons_reference = qty × fuel_reference.average
total_fuel_reference = qty × PA × UA × EWH × fuel_reference.average
fuel_ratio_reference = round(total_fuel_reference / total_mine_prod_bcm, 4)
```

Default: **PA=0.90**, **UA=0.53**, **EWH=4121.0**, dan **total_mine_prod_bcm=91276500.0**.

Actual operational

Jika **fuel_consumed_liters** dan **operating_hours** keduanya tidak null:

```
fuel_cons_actual = fuel_consumed_liters / operating_hours
fuel_cons_lhr = fuel_cons_actual
total_fuel_liters = fuel_consumed_liters
fuel_ratio_actual = round(total_fuel_liters / total_mine_prod_bcm, 4)
fuel_ratio = fuel_ratio_actual
data_source = OPERATIONAL_ACTUAL
```

Jika actual tidak tersedia, **total_fuel_liters**, **fuel_cons_lhr**, dan **fuel_ratio** memakai formula reference serta **data_source=OEM_REFERENCE**.

PA, **UA**, **EWB**, dan **total_mine_prod_bcm** harus lebih besar dari 0. **PA** dan **UA** tidak boleh lebih besar dari 1.

Dewatering

Rumus sama dengan Supporting. Default yang berbeda:

```
PA = 0.90
UA = 0.63
EWB = 4899.0
total_mine_prod_bcm = 91276500.0
```

Monitoring summary

Loading dan Hauling memakai agregasi:

```
fuel_total = Σ(row.qty × row.fuelConsumption)
productivity_total = Σ(row.qty × row.productivity)
actual_fr = fuel_total / productivity_total
```

Supporting dan Dewatering memakai:

```
actual_fr = Σ(row.qty × row.EWB × row.fuelConsumption) /
TOTAL_ANNUAL_PRODUCTION_BCM
```

Variance:

```
variance_pct = round((actual_fr - target_spo_fr) / target_spo_fr × 100, 2)
```

Trend mengelompokkan summary berdasarkan `created_at`, menggabungkan fuel dan production pada tanggal yang sama, lalu mengambil maksimal 21 tanggal terbaru. Sistem tidak membuat titik trend sintetis.

SPO alignment

```
overall_target_spo_fr = Σ(target_spo_fr aktivitas)
target_spo_fuel = actual_production × overall_target_spo_fr
fuel_variance = actual_fuel - target_spo_fuel
fuel_variance_pct = fuel_variance / target_spo_fuel × 100
cost_impact_idr = fuel_variance × fuel_price_per_liter
production_gap = target_production - actual_production
required_production = actual_fuel / overall_target_spo_fr
```

Kondisi	Status
<code>fuel_variance > 0</code>	OVER_BUDGET
<code>fuel_variance < 0</code>	HIGHLY_EFFICIENT
<code>fuel_variance == 0</code>	ALIGNED

Pembulatan dan validasi

- Loading dan Hauling `fuel_ratio` dibulatkan 2 desimal.
- Supporting dan Dewatering `fuel_ratio` dibulatkan 4 desimal.
- Actual Loading hanya dipakai jika liters dan operating hours tersedia serta positif.
- Actual Supporting/Dewatering dipakai jika kedua field tidak null; schema tetap mewajibkan operating hours positif.
- Pembagian dengan productivity atau production nol ditolak atau menghasilkan error validasi.
- Nilai payload divalidasi Pydantic sebelum service dijalankan.

Sumber implementasi

- `backend/app/services/loading_service.py`
- `backend/app/services/hauling_service.py`
- `backend/app/services/supporting_service.py`
- `backend/app/services/dewatering_service.py`
- `backend/app/services/monitoring_service.py`
- `backend/app/services/alignment_service.py`